esprit Se former autrement	Année Universitaire : 2026-2027 **Ateliers SOA** **UP_WEB**

Atelier REST n°5

Sécuriser un service web RESTful (JWT + Refresh Token)

Objectifs

Cet atelier montre comment sécuriser un service web RESTful JAX-RS avec des jetons JWT : un **access token** de 15 minutes pour appeler les ressources, puis un **refresh token** de 7 jours pour en obtenir un nouveau sans ressaisir le mot de passe.

À la fin de l'atelier, vous saurez :

- générer et valider un jeton JWT avec la bibliothèque jjwt ;
- protéger des ressources avec un filtre JAX-RS et l'annotation `@Secured` ;
- renouveler un jeton expiré avec un refresh token, puis le révoquer (déconnexion).

Le projet à sécuriser est celui de la gestion des options et des étudiants (ressources OptionRessources et EtudiantRessources). L'atelier se déroule en trois parties. Le projet compile et se teste à la fin de chacune.

Partie	Contenu	Ce que vous pouvez tester
A	Authentification et jeton d'accès	Obtenir un JWT avec Postman
B	Sécurisation des ressources	401 sans jeton, 201 avec un jeton valide
C	Refresh token	Renouvellement du jeton et déconnexion

Toute la logique JWT tient dans **une seule classe de service**, `JwtService`, que vous créez en partie A et complétez en partie C.

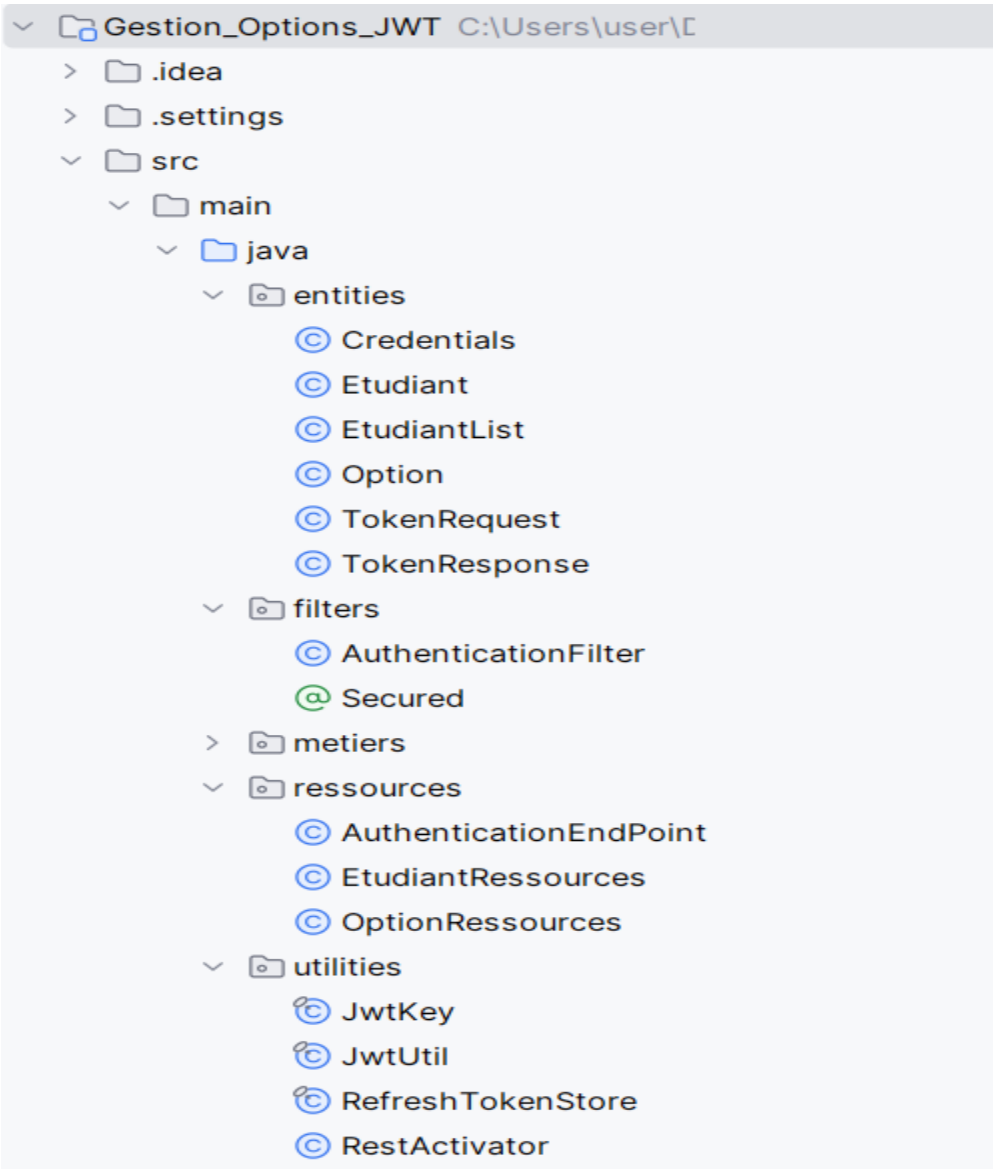
Mise en place de l'environnement

Étape 1 – Ajouter les dépendances. Dans le fichier `pom.xml` du projet, ajoutez, à côté des dépendances Jersey déjà présentes, la bibliothèque JWT, puis rechargez Maven :

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.6.0</version>
    <exclusions>
```

```
        <!-- on garde jackson-databind fourni par jersey-media-json-jackson -->
        <exclusion>
            <groupId>com.fasterxml.jackson.core</groupId>
            <artifactId>jackson-databind</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

Étape 2 – Vérifier la structure du projet. Les packages entities, metiers, ressources et utilities doivent exister (créez ceux qui manquent). Vous créerez vous-même le package filter en partie B. Les fichiers marqués ★ sont ceux que vous créerez pendant l'atelier :



Principe

Le serveur ne garde pas de session : le client prouve son identité à chaque requête grâce à un jeton signé. Le déroulement est le suivant:

1. Le client envoie son login et son mot de passe au serveur.
2. Si l'authentification est valide, le serveur génère un jeton (JWT) pour cet utilisateur.
3. Le client conserve ce jeton. Il peut appeler le service tant que le jeton est valide.
4. À chaque requête, le client envoie le jeton dans l'en-tête `Authorization: Bearer <jeton>`.
5. Le serveur vérifie le jeton. Il accepte la requête si le jeton est valide (signature correcte, non expiré) et répond **401 Unauthorized** sinon.

Un access token vit peu de temps (15 minutes) pour limiter les dégâts en cas de vol. Pour éviter de redemander le mot de passe toutes les 15 minutes, le serveur remet aussi un **refresh token**, valable 7 jours, qui sert uniquement à obtenir un nouvel access token .

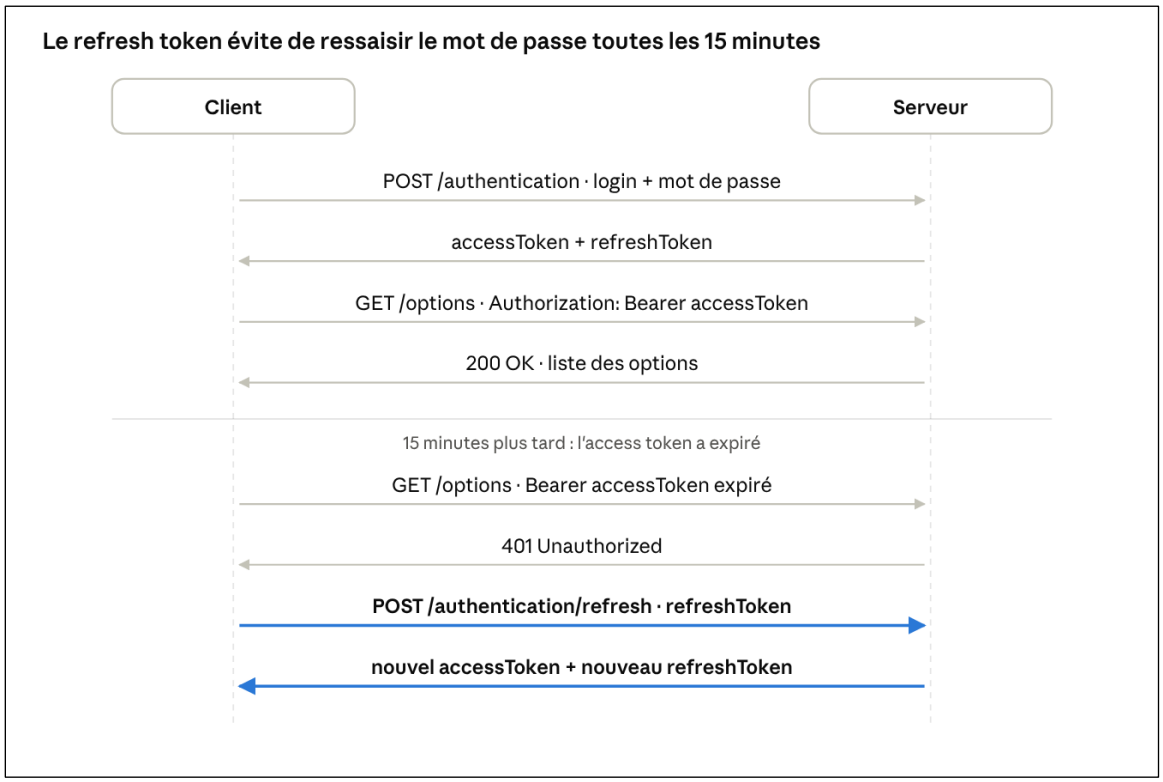


Figure1 : Diagramme de séquence Access-Refresh Token

Partie A – Authentification et génération du jeton d'accès

Objectif : le client envoie son login et son mot de passe, et reçoit un JWT valable 15 minutes. À la fin de cette partie, vous obtenez un jeton avec Postman.

Étape A1 – Créer la classe JwtService dans le package utilities

Clic droit sur le package utilities, puis **New → Java Class**, nom : JwtService. Collez le code suivant :

```
package utilities;

import java.nio.charset.StandardCharsets;
import java.util.Date;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtBuilder;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

public class JwtService {

    // Clé secrète connue uniquement du serveur (HS512 => au moins 64 caractères)
    private static final byte[] SECRET_KEY =
        "gestion-options-esprit-secret-key-change-me-0123456789-
abcdefghijklmnopqrstuvwxyz"
            .getBytes(StandardCharsets.UTF_8);

    // Durée de vie de l'access token
    public static final long ACCESS_MINUTES = 15L;

    // ===== Génération =====

    public static String generateAccessToken(String username, String issuer) {
        long duree = ACCESS_MINUTES * 60 * 1000;   // en millisecondes
        return buildToken(username, issuer, "access", duree, null);
    }

    private static String buildToken(String username, String issuer, String type,
                                     long dureeMillis, String jti) {
        long now = System.currentTimeMillis();
        JwtBuilder builder = Jwts.builder()
                .setSubject(username)                       // sub : l'utilisateur
                .setIssuer(issuer)                          // iss : l'émetteur
                .setIssuedAt(new Date(now))                 // iat : date d'émission
                .setExpiration(new Date(now + dureeMillis))  // exp : date
d'expiration
                .claim("type", type);                       // notre propre claim
        if (jti != null) {
            builder.setId(jti);                             // jti : identifiant
unique (partie C)
        }
        return builder.signWith(SignatureAlgorithm.HS512, SECRET_KEY).compact();
    }

    // ===== Validation =====

    // Lève une exception si la signature est invalide ou si le jeton a expiré
    private static Claims parse(String token) {
        return
```

```
Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJws(token).getBody();
    }

    // Valide un access token : signature, expiration et type
    public static Claims validateAccessToken(String token) {
        Claims claims = parse(token);
        if (!"access".equals(claims.get("type"))) {
            throw new IllegalArgumentException("Ce jeton n'est pas un access token");
        }
        return claims;
    }
}
```

Explications

- SECRET_KEY sert à signer les jetons. Seul le serveur la connaît : sans elle, personne ne peut fabriquer un jeton accepté. L'algorithme HS512 exige au moins 64 caractères. Cette clé ne se met jamais dans le code source.
- buildToken assemble le JWT : les claims sub, iss, iat, exp, puis un claim personnalisé type. signWith(HS512, ...) ajoute la signature et compact() produit la chaîne finale.
- Le claim type indique « access ». Il sera utile en partie C, pour distinguer un access token d'un refresh token. Le paramètre jti aussi : laissez-le à null pour l'instant.
- parse vérifie la signature et la date d'expiration. Si l'un des deux est faux, il lève une exception.
- validateAccessToken appelle parse, puis vérifie que le jeton est bien de type « access ».

Étape A2 – Créer la classe Credentials dans le package entities

Clic droit sur le package entities, puis **New → Java Class**, nom : Credentials. Collez le code suivant :

```
package entities;

import java.io.Serializable;

public class Credentials implements Serializable {

    private String username;
    private String password;

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

Credentials est une classe simple qui regroupe le login et le mot de passe d'un utilisateur. Le client les enverra sous la forme JSON suivante :

```
{
  "username": "Etu001",
```

```
  "password": "pwd001"
}
```

Étape A3 – Créer la ressource AuthenticationEndPoint dans le package ressources

Clic droit sur le package ressources, puis **New → Java Class**, nom : **AuthenticationEndPoint**. Collez le code suivant :

```
package ressources;

import javax.ws.rs.Consumes;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Context;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.UriInfo;

import entities.Credentials;
import utilities.JwtService;

@Path("authentication")
public class AuthenticationEndPoint {

    @Context
    private UriInfo uriInfo;

    @POST
    @Produces(MediaType.TEXT_PLAIN)
    @Consumes(MediaType.APPLICATION_JSON)
    public Response authenticateUser(Credentials cred) {
        try {
            // 1. Vérifier les identifiants
            authenticate(cred.getUsername(), cred.getPassword());

            // 2. Générer le jeton pour cet utilisateur
            String issuer = uriInfo.getBaseUri().toString();
            String token = JwtService.generateAccessToken(cred.getUsername(),
issuer);

            // 3. Renvoyer le jeton au client
            return Response.ok(token).build();

        } catch (Exception e) {
            return Response.status(Response.Status.FORBIDDEN).build();
        }
    }

    private void authenticate(String username, String password) {
                if (username == null || username.trim().isEmpty()
                || password == null || password.trim().isEmpty()) {
            throw new SecurityException("Identifiants invalides");
```

```
        }
    }
}
```

Explications

- @Path("authentication") rend la ressource accessible à l'adresse .../rest/authentication. Elle est publique : on ne lui met pas @Secured, car le client n'a pas encore de jeton.
- @Consumes(APPLICATION_JSON) indique que le corps de la requête est du JSON. JAX-RS le convertit en objet Credentials, passé en paramètre de la méthode.
- @Context UriInfo donne accès à l'adresse du service, utilisée comme émetteur (iss) du jeton.
- authenticate lève une exception si les identifiants sont invalides (ou si le corps de la requête est absent). La réponse est alors **403 Forbidden**.
- Sinon, generateAccessToken crée le jeton et on le renvoie en texte brut avec **200 OK**.

Étape A4 – Tester l'authentification avec Postman

Déployez le projet sur le serveur, puis dans Postman :

1. Sélectionnez la méthode **POST**.
2. Saisissez l'URL : http://localhost:8080/Gestion_Options_Etudiants_war_exploded/rest/authentication
3. Ouvrez l'onglet **Body**, choisissez **raw**, puis le type **JSON** dans la liste déroulante.
4. Saisissez les identifiants d'un étudiant, puis cliquez sur **Send** :

```
{
  "username": "I001",
  "password": "pwd001"
}
```

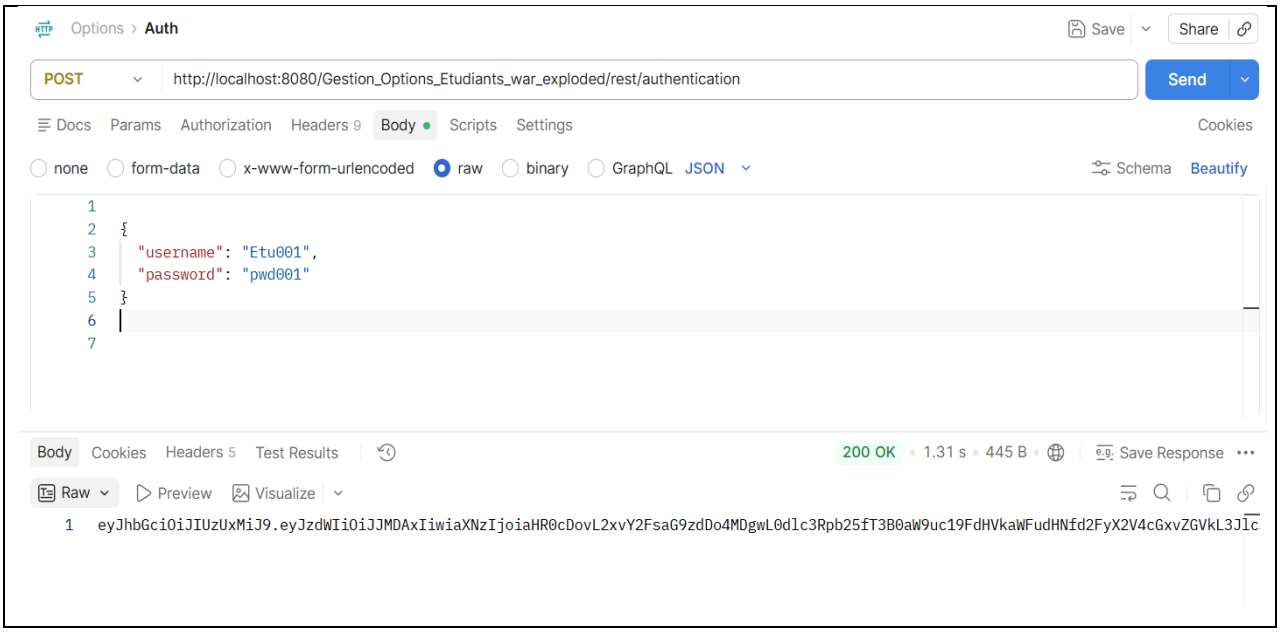
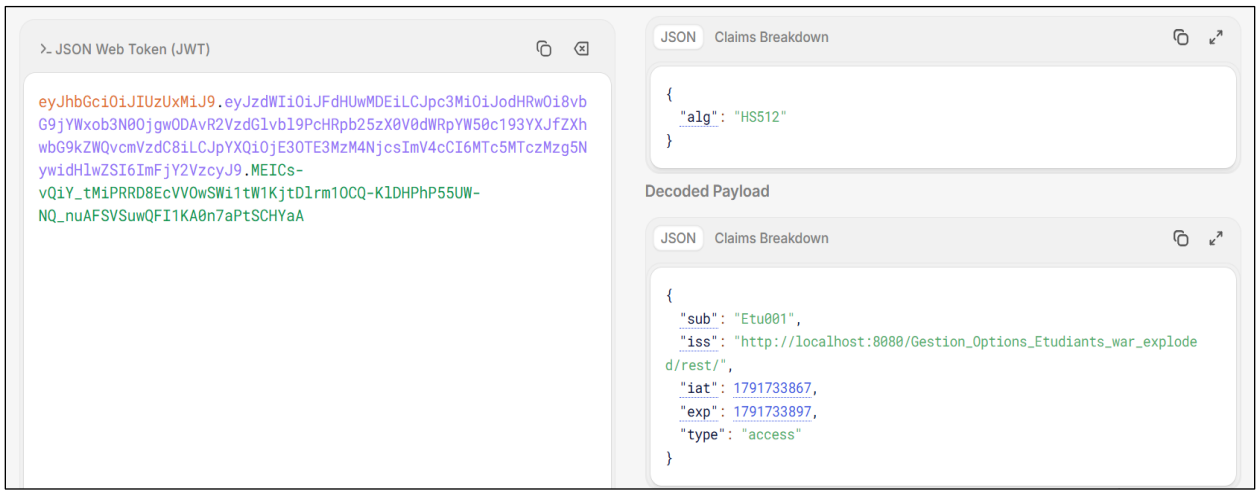


Figure2 : Authentication

Étape A5 – Analyser le contenu du jeton sur jwt.io

Un JWT est un objet JSON compressé (d'où son nom). Il contient trois parties séparées par des points : l'en-tête, le contenu (*payload*) et la signature, chacune encodée en Base64URL.

Ouvrez le site jwt.io et collez votre jeton dans la zone *Encoded*. Le contenu doit ressembler à ceci :



Partie B – Sécurisation des ressources

Objectif : seules les requêtes qui portent un jeton valide peuvent appeler une ressource sécurisée. Le client envoie son jeton dans l'en-tête Authorization. Il faut donc un **filtre**, appliqué à chaque appel d'une ressource sécurisée, et une **annotation** @Secured qui indique quelles ressources passent par ce filtre.

Étape B1 – Créer le package filter et l'annotation Secured

Créez d'abord le package filter : clic droit sur le dossier java, puis **New → Package**. Clic droit sur ce package, puis **New → Java Class**, choisissez le type **Annotation** et nommez-la Secured. Collez le code suivant :

```
package filter;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import javax.ws.rs.NameBinding;

@NameBinding
@Retention(RetentionPolicy.RUNTIME)
@Target({ ElementType.TYPE, ElementType.METHOD })
public @interface Secured {
}
```

Explications

- @Secured n'est pas une annotation prédéfinie : nous la créons nous-mêmes (peut être appeler autrement).
- @NameBinding est une méta-annotation JAX-RS. Elle lie un filtre à toutes les classes et méthodes qui portent l'annotation.
- @Target({ TYPE, METHOD }) autorise @Secured sur une classe entière ou sur une seule méthode.

Étape B2 – Créer le filtre AuthenticationFilter dans le package filter

Clic droit sur le package filter, puis **New → Java Class**, nom : AuthenticationFilter. Collez le code suivant :

```
package filter;

import java.io.IOException;

import javax.annotation.Priority;
import javax.ws.rs.Priorities;
import javax.ws.rs.container.ContainerRequestContext;
import javax.ws.rs.container.ContainerRequestFilter;
import javax.ws.rs.core.HttpHeaders;
import javax.ws.rs.core.Response;
```

```
import javax.ws.rs.ext.Provider;

import utilities.JwtService;

@Secured
@Provider
@Priority(Priorities.AUTHENTICATION)
public class AuthenticationFilter implements ContainerRequestFilter {

    private static final String AUTHENTICATION_SCHEME = "Bearer";

    @Override
    public void filter(ContainerRequestContext requestContext) throws IOException {

        // 1. Lire l'en-tête Authorization de la requête
        String authorizationHeader =
                requestContext.getHeaderString(HttpHeaders.AUTHORIZATION);

        // 2. Il doit exister et commencer par "Bearer "
        if (authorizationHeader == null
                || !authorizationHeader.startsWith(AUTHENTICATION_SCHEME + " ")) {
            abortWithUnauthorized(requestContext);
            return;
        }

        // 3. Extraire le jeton (ce qui suit "Bearer")
        String token =
authorizationHeader.substring(AUTHENTICATION_SCHEME.length()).trim();

        // 4. Valider le jeton : sinon, refuser la requête
        try {
            JwtService.validateAccessToken(token);
        } catch (Exception e) {
            abortWithUnauthorized(requestContext);
        }
    }

    private void abortWithUnauthorized(ContainerRequestContext requestContext) {
        // Arrête le traitement et renvoie 401 Unauthorized
        requestContext.abortWith(
                Response.status(Response.Status.UNAUTHORIZED)
                        .header(HttpHeaders.WWW_AUTHENTICATE, AUTHENTICATION_SCHEME)
                        .build());
    }
}
```

Explications

- Un filtre qui implémente ContainerRequestFilter intercepte la requête **avant** l'appel de la ressource. Sa méthode filter est exécutée à chaque requête concernée.
- @Secured sur le filtre le lie aux ressources annotées @Secured. @Provider l'enregistre auprès de JAX-RS. @Priority(AUTHENTICATION) le place au bon moment dans la chaîne de traitement.

- Le filtre suit quatre étapes : lire l'en-tête, vérifier qu'il commence par Bearer, extraire le jeton, puis le valider avec JwtService.validateAccessToken.
- abortWith(...) interrompt la requête : la ressource n'est jamais appelée et le client reçoit **401 Unauthorized**.

Étape B3 – Sécuriser OptionRessources et EtudiantRessources dans le package ressources

Ouvrez la classe OptionRessources et ajoutez @Secured au-dessus de @Path, sur la **classe**. Toutes ses méthodes (liste, recherche par code, ajout, modification, suppression) exigent alors un jeton valide.

Remarque :

@Secured peut se placer à deux endroits.

- **Au-dessus de @Path**, sur la classe : toutes les méthodes sont sécurisées.
- **Au-dessus d'une seule méthode** (par exemple **@GET** ou **@POST**) : **seule** celle-ci l'est, les autres restent publiques.

```
package ressources;

import filter.Secured;
// ... autres imports inchangés

@Secured                      // AJOUT : toutes les méthodes de la classe sont
protégées
@Path("options")
public class OptionRessources {

    public static OptionBusiness optionMetier = new OptionBusiness();

    // getOptions (GET), getOptionById (GET), addOption (POST),
    // updateOption (PUT), deleteOption (DELETE) : inchangées
}
```

Étape B4 – Tester tous les scénarios avec Postman

Redéployez le projet. Pour chaque test, envoyez une requête **GET** vers http://localhost:8080/<nom-du-projet>/rest/options (aucun corps n'est nécessaire). Le jeton se place dans l'onglet **Authorization** de Postman, ce qui produit l'en-tête suivant :

```
GET http://localhost:8080/<nom-du-projet>/rest/options
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIi...
```

N°	Scénario	Comment le faire dans Postman	Résultat attendu
1	Sans jeton	Onglet **Authorization** → *No Auth*	**401 Unauthorized**

N°	Scénario	Comment le faire dans Postman	Résultat attendu
2	Avec le jeton valide	**Authorization** → *Bearer Token*, collez le jeton de l'étape A4	**200 OK** et la liste JSON des options (404 si la liste est vide)
3	Avec un jeton invalide	Modifiez une lettre du jeton	**401 Unauthorized**
4	Avec un jeton expiré	Mettez `ACCESS_MINUTES = 1` dans `JwtService`, redéployez, générez un jeton, attendez 1 minute et réessayez	**401 Unauthorized**
5	Autre ressource sécurisée	`GET /rest/etudiants/I001`, d'abord sans jeton, puis avec le jeton valide	**401 Unauthorized**, puis **200 OK** (404 si l'étudiant n'existe pas)
6	Ressource publique	`POST /rest/authentication` sans jeton	**200 OK** : seule l'authentification reste accessible sans jeton

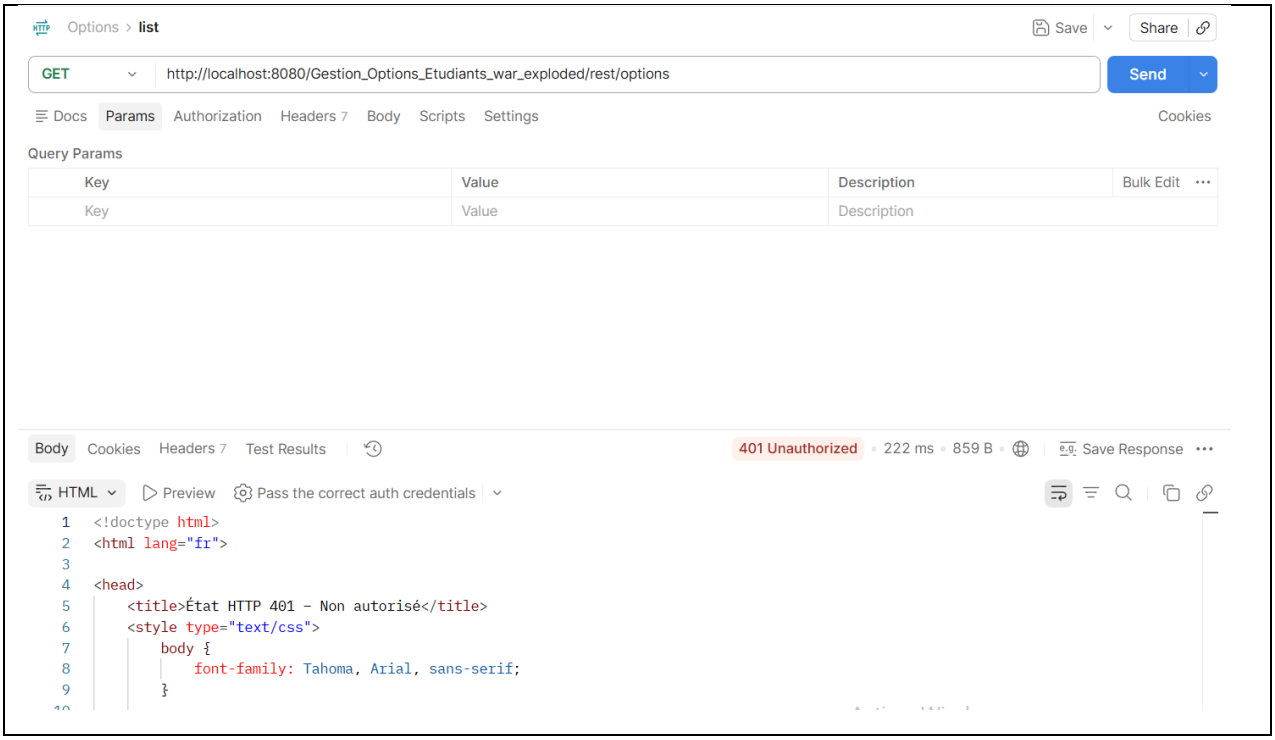


Figure 3: Sans jeton

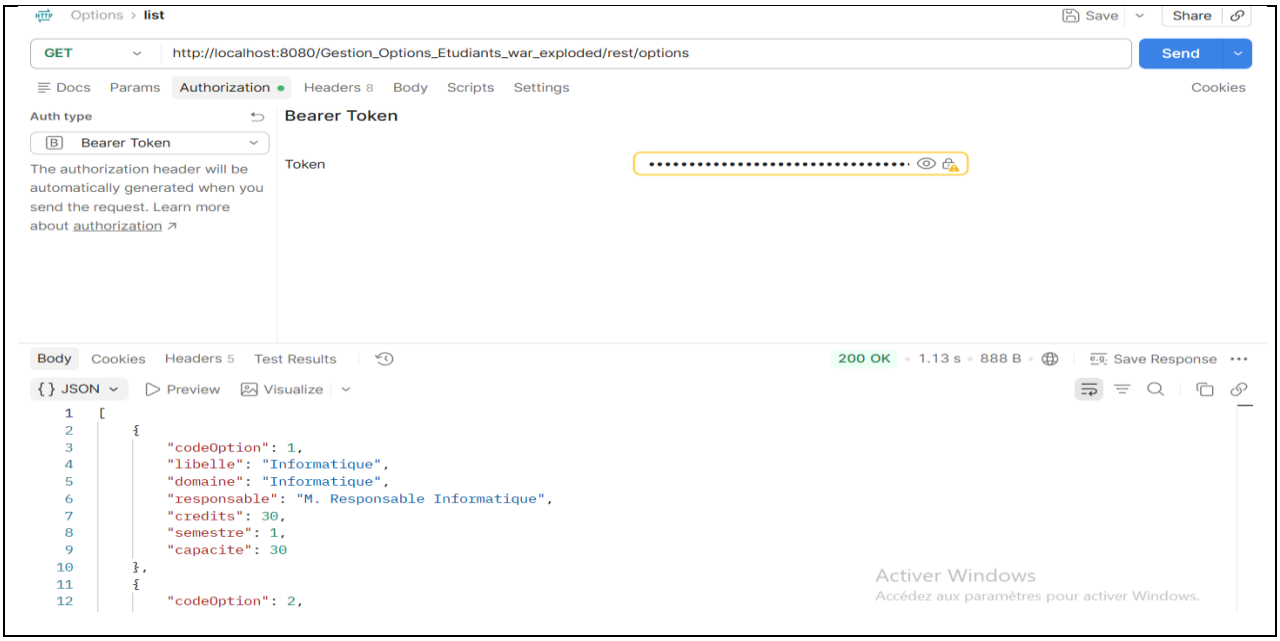


Figure 4 : avec jeton

Partie C – Rafraîchissement du jeton (Refresh Token)

Problème : l'access token expire au bout de 15 minutes, et l'utilisateur devrait alors ressaisir son login et son mot de passe. Solution : le serveur remet, à la connexion, un second jeton de plus longue durée, le **refresh token**. Il sert uniquement à obtenir un nouvel access token.

	Access token	Refresh token
Durée de vie	15 minutes	7 jours
Usage	Appeler les ressources sécurisées	Obtenir un nouvel access token
Envoyé à	Chaque ressource, dans Authorization: Bearer	/authentication/refresh uniquement
Mémorisé par le serveur	Non	Oui, pour pouvoir le révoquer
Claim type	access	refresh

Le scénario complet est le suivant :

1. Le client se connecte et reçoit un access token **et** un refresh token.
2. Il appelle les ressources avec l'access token.
3. Après 15 minutes, une ressource répond **401** : l'access token a expiré.
4. Le client envoie son refresh token à /authentication/refresh.

5. Le serveur vérifie ce jeton, le révoque, et renvoie un nouveau couple de jetons.
6. À la déconnexion, `/authentication/logout` révoque le refresh token.

Pourquoi le serveur mémorise-t-il les refresh tokens ? Un JWT signé ne peut pas être annulé avant sa date d'expiration. Le serveur enregistre donc l'identifiant unique (`jti`) de chaque refresh token émis, et n'accepte un refresh token que si son `jti` est encore dans ce registre. **Supprimer le `jti` révoque le jeton.** Chaque refresh token est à usage unique : c'est la **rotation**.

Étape C1 – Compléter la classe `JwtService` du package `utilities`

Ouvrez `JwtService`. D'abord, ajoutez ces trois imports en haut du fichier :

```
import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
```

Puis ajoutez le code suivant **à la fin de la classe**, juste avant la dernière accolade :

```
    // ===== Partie C : refresh token =====

    // Durée de vie du refresh token
    public static final long REFRESH_DAYS = 7L;

    // Registre des refresh tokens valides : jti -> utilisateur
    // (en mémoire pour l'atelier ; en production : table en base de données)
    private static final Map<String, String> REFRESH_STORE = new
ConcurrentHashMap<>();

    // Génère un refresh token et l'enregistre dans le registre
    public static String generateRefreshToken(String username, String issuer) {
        String jti = UUID.randomUUID().toString();          // identifiant unique
        long duree = REFRESH_DAYS * 24 * 60 * 60 * 1000;     // en millisecondes
        String token = buildToken(username, issuer, "refresh", duree, jti);
        REFRESH_STORE.put(jti, username);
        return token;
    }

    // Valide un refresh token ET le consomme (usage unique). Retourne l'utilisateur.
    public static String useRefreshToken(String token) {
        Claims claims = parse(token);
        if (!"refresh".equals(claims.get("type"))) {
            throw new IllegalArgumentException("Ce jeton n'est pas un refresh
token");
```

```
        }
        // remove() supprime l'entrée et renvoie null si le jti est inconnu
        if (REFRESH_STORE.remove(claims.getId()) == null) {
            throw new IllegalArgumentException("Refresh token révoqué ou déjà
utilisé");
        }
        return claims.getSubject();
    }

    // Révoque un refresh token (déconnexion)
    public static void revokeRefreshToken(String token) {
        try {
            REFRESH_STORE.remove(parse(token).getId());
        } catch (Exception e) {
            // jeton déjà invalide ou expiré : il n'y a rien à révoquer
        }
    }
```

Explications

- `generateRefreshToken` réutilise `buildToken` de la partie A, cette fois avec le type `refresh`, 7 jours de durée et un `jti` aléatoire. Le `jti` est ensuite mémorisé dans `REFRESH_STORE`.
- `REFRESH_STORE` est un simple dictionnaire en mémoire. `ConcurrentHashMap` supporte plusieurs requêtes simultanées. Les entrées sont perdues au redémarrage du serveur : les utilisateurs doivent alors se reconnecter.
- `useRefreshToken` fait trois contrôles : signature et expiration (`parse`), type `refresh`, puis présence du `jti` dans le registre. `remove` supprime l'entrée : le jeton ne peut plus servir une seconde fois.
- Le claim `type` joue dans les deux sens : un refresh token présenté à une ressource est refusé par `validateAccessToken`, et un access token présenté à `/refresh` est refusé par `useRefreshToken`.
- `revokeRefreshToken` supprime le `jti` du registre. C'est la déconnexion.

Étape C2 – Créer la classe `TokenResponse` dans le package `entities`

Clic droit sur le package `entities`, puis **New → Java Class**, nom : `TokenResponse`. Collez le code suivant :

```
package entities;

import java.io.Serializable;

public class TokenResponse implements Serializable {

    private String accessToken;
    private String refreshToken;
    private String tokenType = "Bearer";
    private long expiresIn;   // durée de l'access token, en secondes

    public TokenResponse() {
    }
```

```
    public TokenResponse(String accessToken, String refreshToken, long expiresIn) {
        this.accessToken = accessToken;
        this.refreshToken = refreshToken;
        this.expiresIn = expiresIn;
    }

    public String getAccessToken() { return accessToken; }
    public void setAccessToken(String accessToken) { this.accessToken = accessToken;
}

    public String getRefreshToken() { return refreshToken; }
    public void setRefreshToken(String refreshToken) { this.refreshToken =
refreshToken; }

    public String getTokenType() { return tokenType; }
    public void setTokenType(String tokenType) { this.tokenType = tokenType; }

    public long getExpiresIn() { return expiresIn; }
    public void setExpiresIn(long expiresIn) { this.expiresIn = expiresIn; }
}
```

Cet objet est transformé en JSON par JAX-RS. Il permet de renvoyer les deux jetons dans une seule réponse, au lieu d'un simple texte.

Étape C3 – Remplacer le contenu de AuthenticationEndPoint dans le package ressources

Ouvrez AuthenticationEndPoint et **remplacez toute la classe** par le code suivant. Les parties modifiées ou ajoutées sont signalées par NOUVEAU ou MODIFIÉ.

```
package ressources;

import javax.ws.rs.Consumes;
import javax.ws.rs.FormParam;
import javax.ws.rs.POST;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.Context;
import javax.ws.rs.core.MediaType;
import javax.ws.rs.core.Response;
import javax.ws.rs.core.UriInfo;

import entities.Credentials;
import entities.TokenResponse;
import utilities.JwtService;

@Path("authentication")
public class AuthenticationEndPoint {

    @Context
    private UriInfo uriInfo;
```

```
    // ===== 1. Login : MODIFIÉ
    @POST
    @Produces(MediaType.APPLICATION_JSON)
    @Consumes(MediaType.APPLICATION_JSON)
    public Response authenticateUser(Credentials cred) {
        try {
            authenticate(cred.getUsername(), cred.getPassword());
            return Response.ok(buildTokens(cred.getUsername())).build();
        } catch (Exception e) {
            return Response.status(Response.Status.FORBIDDEN).build();
        }
    }

    // ===== 2. Refresh
    @POST
    @Path("refresh")
    @Produces(MediaType.APPLICATION_JSON)
    @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
    public Response refresh(@FormParam("refreshToken") String refreshToken) {
        try {
            // Valide le refresh token et le consomme (rotation)
            String username = JwtService.useRefreshToken(refreshToken);
            // Renvoie un nouveau couple de jetons
            return Response.ok(buildTokens(username)).build();
        } catch (Exception e) {
            return Response.status(Response.Status.UNAUTHORIZED).build();
        }
    }

    // 3. Logout
    @POST
    @Path("logout")
    @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
    public Response logout(@FormParam("refreshToken") String refreshToken) {
        JwtService.revokeRefreshToken(refreshToken);
        return Response.noContent().build();
    }



    // génère les deux jetons et les place dans un TokenResponse
    private TokenResponse buildTokens(String username) {
        String issuer = uriInfo.getBaseUri().toString();
        String accessToken = JwtService.generateAccessToken(username, issuer);
        String refreshToken = JwtService.generateRefreshToken(username, issuer);
        return new TokenResponse(accessToken, refreshToken, JwtService.ACCESS_MINUTES
* 60);
    }

    private void authenticate(String username, String password) {
        if (username == null || username.trim().isEmpty()
                || password == null || password.trim().isEmpty()) {
            throw new SecurityException("Identifiants invalides");
        }
```

```
    }
}
```

Explications

- Le login renvoie maintenant du **JSON** (TokenResponse) au lieu d'un texte, grâce à buildTokens.
- /authentication/refresh n'a pas besoin de login ni de mot de passe : le refresh token est la preuve que l'utilisateur s'est déjà authentifié. Si le jeton est invalide, expiré, révoqué ou déjà utilisé, la réponse est **401**.
- /authentication/logout révoque le refresh token et répond **204 No Content**. L'access token en cours reste valable jusqu'à son expiration (15 minutes au maximum).
- Ces deux points d'accès ne portent pas @Secured : ils doivent rester accessibles quand l'access token a expiré.

Étape C4 – Tester le scénario complet avec Postman

Redéployez le projet, puis enchaînez les tests dans l'ordre. Le login utilise **Body → raw → JSON**, comme à l'étape A4. Les requêtes refresh et logout utilisent **Body → x-www-form-urlencoded** avec la clé refreshToken.

N°	Scénario	Requête	Résultat attendu
1	Connexion	POST /rest/authentication avec le corps JSON {"username": "I001", "password": "pwd001"}	**200 OK** et un JSON avec accessToken et refreshToken
2	Ressource sécurisée	GET /rest/options avec *Bearer Token* = accessToken	**200 OK** et la liste des options
3	Access token expiré	Mettez ACCESS_MINUTES = 1, redéployez, reconnectez-vous, attendez 1 minute, rappelez la ressource	**401 Unauthorized**
4	Rafraîchissement	POST /rest/authentication/refresh avec refreshToken = valeur reçue au test 1 (ou 3)	**200 OK** et un **nouveau** couple de jetons
5	Nouveau jeton	Rappelez GET	**200 OK**

N°	Scénario	Requête	Résultat attendu
		`/rest/options` avec le nouvel `accessToken`	
6	Rejeu de l'ancien refresh token	Renvoyez le refresh token du test 4	**401 Unauthorized** (rotation)
7	Refresh token utilisé comme access token	Collez un `refreshToken` dans *Bearer Token* sur `GET /rest/options`	**401 Unauthorized**
8	Déconnexion	POST `/rest/authentication/logout` avec le dernier `refreshToken`, puis `/refresh` avec le même jeton	**204 No Content**, puis **401 Unauthorized**

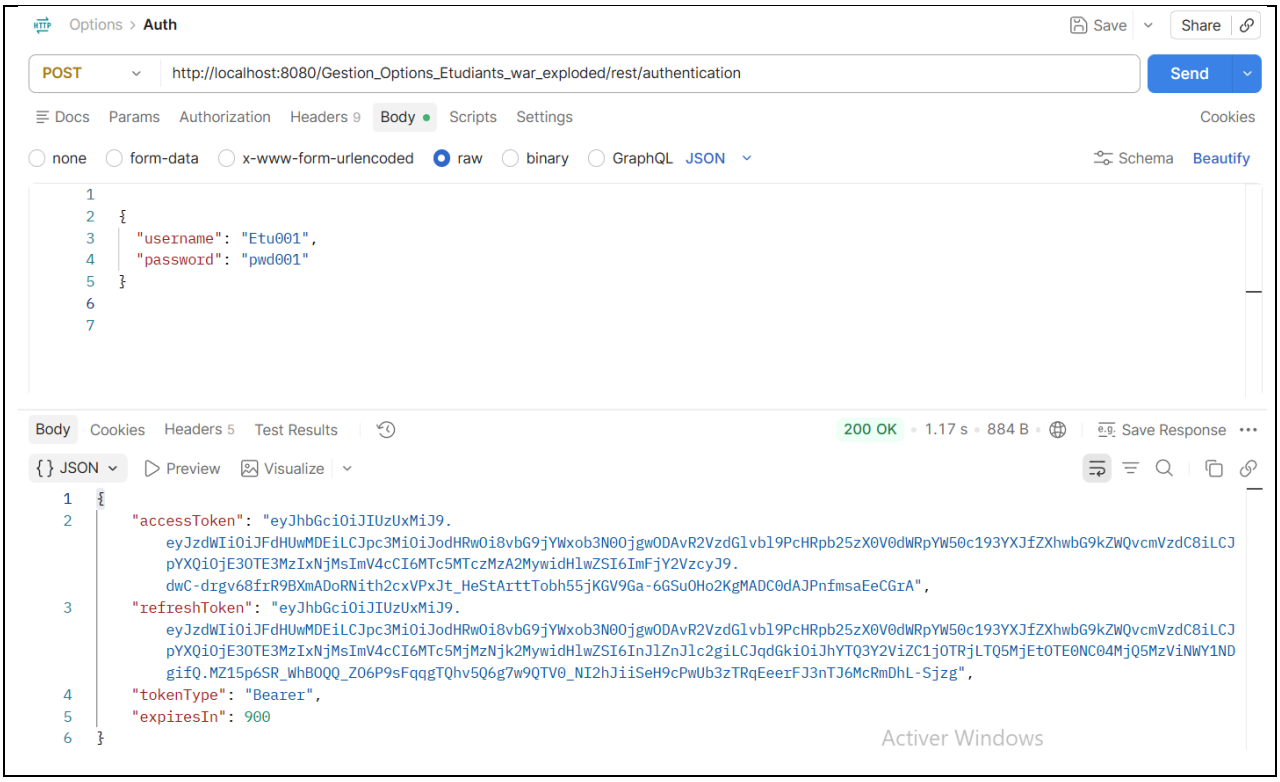


Figure 5 : Authentication with refresh Token

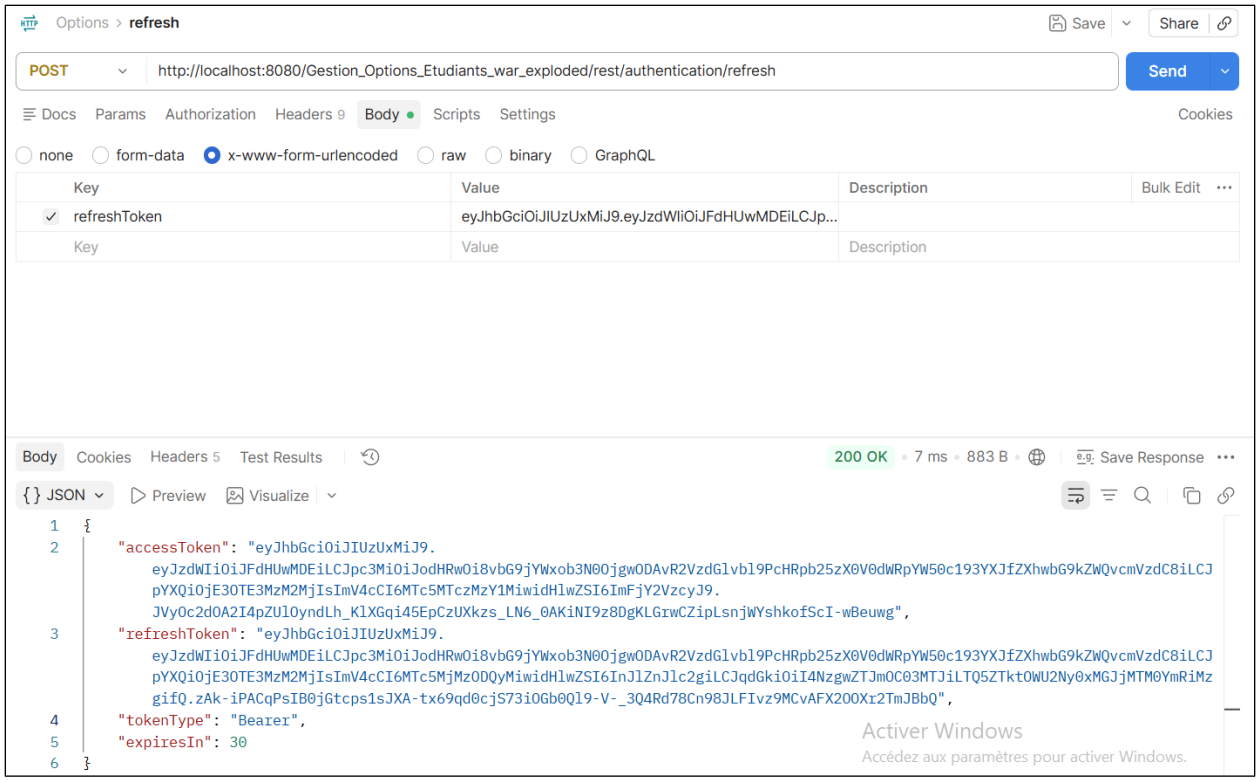


Figure 6 : Rafraichissement

Récapitulatif :

L'atelier demande de créer six classes. Une seule contient la logique JWT.

Classe	Package	Créée en	Rôle
JwtService	utilities	A1, complétée en C1	Génère, valide et révoque les jetons
Credentials	entities	A2	Login et mot de passe envoyés en JSON
AuthenticationEndPoint	ressources	A3, remplacée en C3	Login, refresh et logout
Secured	filter	B1	Annotation qui désigne les ressources protégées
AuthenticationFilter	filter	B2	Vérifie l'access token de chaque requête protégée

Classe	Package	Créée en	Rôle
TokenResponse	entities	C2	Réponse JSON contenant les deux jetons